

112590024 邱紹澹 作業系統作業 HW4

10.21: Assume that we have a demand-paged memory
The page table is held in registers
It takes 8 milliseconds to service a page fault if an empty frame is available or if the replaced page is not modified and 20 milliseconds if the replaced page is modified
Memory-access time is 100 nanoseconds
Assume that the page to be replaced is modified 70 percent of the time
What is the maximum acceptable page-fault rate for an effective access time of no more than 200 nanoseconds?

10.21

$$EAT = (1-p) \times 100 + p \times (0.3 \times 8,000,000 + 0.7 \times 20,000,000) \leq 200$$

$$100 + p \times 16,399,900 \leq 200$$

$$p \leq 16,399,900 / 100 \approx 6.1 \times 10^{-6}$$

最大可接受分頁錯誤率約為 6.1×10^{-6} ，即每 160,000 次存取最多 1 次分頁錯誤。

10.24: Apply the (1) FIFO (2) LRU (3) Optimal (OPT) replacement algorithms for the following page reference string:

0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7.

Indicate the number of page faults for each algorithm assuming demand paging with three frames

10.24

(1) FIFO

步驟	存取	框架	缺頁
1	0	[0, ,]	✓
2	6	[0, 6, _]	✓
3	3	[0, 6, 3]	✓
4	0	[0, 6, 3]	
5	2	[2, 6, 3]	✓
6	6	[2, 6, 3]	

步驟	存取	框架	缺頁
7	3	[2, 6, 3]	
8	5	[2, 5, 3]	✓
9	2	[2, 5, 3]	
10	4	[2, 5, 4]	✓
11	1	[1, 5, 4]	✓
12	3	[1, 3, 4]	✓
13	0	[1, 3, 0]	✓
14	6	[6, 3, 0]	✓
15	1	[6, 1, 0]	✓
16	4	[6, 1, 4]	✓
17	2	[2, 1, 4]	✓
18	3	[2, 3, 4]	✓
19	5	[2, 3, 5]	✓
20	7	[7, 3, 5]	✓

缺頁次數：16

(2) LRU

步驟	存取	框架	缺頁
1	0	[0, ,]	✓
2	6	[0, 6, _]	✓
3	3	[0, 6, 3]	✓
4	0	[0, 6, 3]	
5	2	[0, 2, 3]	✓
6	6	[0, 2, 6]	✓
7	3	[3, 2, 6]	✓
8	5	[3, 5, 6]	✓
9	2	[3, 5, 2]	✓
10	4	[4, 5, 2]	✓
11	1	[4, 1, 2]	✓
12	3	[4, 1, 3]	✓
13	0	[0, 1, 3]	✓
14	6	[0, 6, 3]	✓
15	1	[1, 6, 3]	✓
16	4	[1, 6, 4]	✓
17	2	[2, 6, 4]	✓

步驟	存取	框架	缺頁
18	3	[2, 6, 3]	✓
19	5	[5, 6, 3]	✓
20	7	[5, 7, 3]	✓

缺頁次數：18

(3) OPT

步驟	存取	框架	缺頁
1	0	[0, ,]	✓
2	6	[0, 6, _]	✓
3	3	[0, 6, 3]	✓
4	0	[0, 6, 3]	
5	2	[2, 6, 3]	✓
6	6	[2, 6, 3]	
7	3	[2, 6, 3]	
8	5	[5, 6, 3]	✓
9	2	[5, 6, 3]	
10	4	[5, 4, 3]	✓
11	1	[1, 4, 3]	✓
12	3	[1, 4, 3]	
13	0	[0, 4, 3]	✓
14	6	[0, 6, 3]	✓
15	1	[1, 6, 3]	✓
16	4	[4, 6, 3]	✓
17	2	[4, 2, 3]	✓
18	3	[4, 2, 3]	
19	5	[5, 2, 3]	✓
20	7	[5, 7, 3]	✓

缺頁次數：16

演算法	缺頁次數
FIFO	16
LRU	18
OPT	16

10.37: What is the cause of thrashing? How does the system detect thrashing? Once it detects thrashing, what can the system do to eliminate this problem?

10.37

- **Frame**→ 實體記憶體
- **Page**→ 邏輯記憶體

原因：行程實際需要的Page數超過分配到的Frame數，導致thrashing。CPU 使用率下降後，OS 誤判需要增加行程，Frame競爭更激烈。

偵測：使用 Working Set Model 追蹤每個行程的工作集（在一段時間 Δ 內使用到的Page集合），若所有行程工作集總和超過可用Frame總數，即發生 Thrashing。也可用 Page Fault Frequency (PFF) 監控，超過上限代表Frame不足。

解決：暫停部分行程並將其頁面 swap out，釋放Frame給其他行程；或直接降低多程式度，等 Thrashing 消除後再逐步恢復行程執行。

11.17: Describe some advantages and disadvantages of using NVM devices as a caching tier and as a disk-drive replacement compared with using only HDDs.

11.17

As a caching tier (快取層)

優點：NVM 速度遠快於 HDD，熱資料 (frequently accessed data) 存在 NVM 可大幅降低存取延遲，同時 HDD 負擔減少、壽命延長。

缺點：NVM 容量小且價格高，快取管理演算法複雜（需決定哪些資料放 NVM），資料在 NVM 與 HDD 之間搬移也有額外開銷。

As a disk-drive replacement (取代 HDD)

優點：無機械結構，存取速度快、延遲低、抗震、耗電少、體積小。

缺點：價格每 GB 比 HDD 貴，寫入次數有上限 (write endurance)，大容量儲存成本仍遠高於 HDD。

雖然現在價格下降很多，但如果資料量很大，也是不曉得一個成本。

11.18: Compare the performance of C-SCAN and SCAN scheduling, assuming a uniform distribution of requests

Consider the average response time (the time between the arrival of a request and the completion of that request's service), the variation in response time, and the effective bandwidth

How does performance depend on the relative sizes of seek time and rotational latency?

11.18

Average response time：兩者相近，但 C-SCAN 單向掃描使等待時間分布更均勻。

Variation：SCAN 兩端請求等待較久，變異大；C-SCAN 變異小、更可預測。

Bandwidth：SCAN 來回都服務，頻寬略優於 C-SCAN (C-SCAN 回程空跑)。

Seek time vs Rotational latency：Seek time 為主時兩者差異明顯；Rotational latency 為主時排程影響變小，瓶頸在等磁碟旋轉而非磁頭移動。

11.21: Compare the throughput achieved by a RAID level 5 organization with that achieved by a RAID level 1 organization.

(a) Read operations on single blocks.

(b) Read operations on multiple contiguous blocks.

11.21

(a)

RAID 1：兩份鏡像都可以讀，可同時從兩顆硬碟處理不同請求，throughput 較高。

RAID 5：資料分散在多顆硬碟，單一 block 可從對應硬碟直接讀取，throughput 與 RAID 1 相近，但多顆硬碟可平行處理更多請求。

(b)

RAID 1：只能從一顆硬碟讀，無法平行讀取，throughput 較低。

RAID 5：資料以 stripe 分散在所有硬碟，讀取連續大量資料時可平行從多顆硬碟同時讀取，throughput 明顯優於 RAID 1。

總結：單一 block 讀取兩者差異不大；大量連續讀取 RAID 5 因 striping 平行讀取，throughput 顯著較高。

14.14: Consider a file system on a disk that has both logical and physical block sizes of 512 bytes

Assume that the information about each file is already in memory
For each of the three allocation strategies (contiguous, linked, and indexed), answer these questions:

(a) How is the logical-to-physical address mapping accomplished in this system? (For indexed allocation, assume that a file is always less than 512 blocks long)

(b) If we are currently at logical block 10 (the last block accessed was block 10) and want to access logical block 4, how many physical blocks must be read from the disk?

14.14

Contiguous

(a) 起始位址 b ，邏輯區塊 $n \rightarrow$ 實體位址 $=b+n$ ，直接計算。

(b) 直接算 $b+4$ ，讀 1個。

Linked

(a) 從第一個區塊開始，沿每個區塊內的指標依序走訪到第 n 個節點。

(b) 無法往回跳，從 block 0 沿指標走到 block 4，讀 5個 (block 0~4)。

Indexed

(a) Index block 存所有指標，邏輯區塊 n 對應 index block 第 n 個指標所指的實體位址。

(b) 先讀 index block，再讀 block 4，讀 2個。

14.15: Consider a file system that uses inodes to represent files
Disk blocks are 8KB in size, and a pointer to a disk block requires 4 bytes
This file system has 12 direct disk blocks, as well as single, double, and triple indirect disk blocks
What is the maximum size of a file that can be stored in this file system?

14.15

每個 block 可存指標數： $8192 \div 4 = 2048$

層級	Block 數
Direct	12
Single indirect	2048
Double indirect	$2048^2 = 4,194,304$
Triple indirect	$2048^3 = 8,589,934,592$

總大小 = $(12 + 2048 + 2048^2 + 2048^3) \times 8KB \approx 64 TB$